

OMPitchField library

Reference

by Paul NAUERT

OMPitchField library: Reference

Edited by Karim Haddad

Published Date: 24/02/2003 17:50

This manual can not be copied, in whole or part, without the written consent of IRCAM.

This manual was written by Paul Nauert, and was produced under the editorial responsibility of Karim Haddad, Marketing and Communication Dept. , IRCAM.

OpenMusic was conceived and programmed by Gérard Assayag and Carlos Agon.

The OMPitchField library was conceived and programmed by Paul Nauert, with the support of a grant from the University of California, Santa Cruz Faculty Senate.

This documentation corresponds to version 1.0 of the library, and to version 4.0 or higher of OpenMusic.

Apple Macintosh™ is a trademark of Apple Computer, Inc.

OpenMusic™ is a trademark of IRCAM.

Copyright © 2003 by IRCAM. All rights reserved.

The use of this software and its documentation is restricted to members of the IRCAM software users' group. For any supplementary information, contact:

Marketing and Communication Dept.,
IRCAM
1 place Igor-Stravinsky
F-75004, Paris
FRANCE
Phone: (33) (1) 44 78 49 62
Fax: (33) (1) 42 77 29 47
Email: palumbo@ircam.fr

Send comments or suggestions to the editor:

Karim Haddad
Marketing and Communication Dept.,
IRCAM
1 place Igor-Stravinsky
F-75004, Paris
FRANCE
Phone: (33) (1) 44 78 16 05
Email: haddad@ircam.fr

For more information:

<http://www.forumnet.ircam.fr>

Table of Contents

1. Introduction	1
1.1. Stage 1. The pcset pool.....	2
1.2. Stage 2. Cyclically generated pitch fields	2
1.3. Stage 3. Searching the field	3
1.4. Stage 4. Processing the search results	4
1.5. On inclusion and similarity.....	5
1.6. Where to go from here.....	6
I. OMPitchField Reference	1
list-ti-primeforms	1
list-t-primeforms.....	2
t-primeform	3
ti-primeform	4
make-t-setclass	5
make-ti-setclass.....	6
set-complement.....	7
make-pfield	8
p->mc.....	10
mc->p.....	11
find-pc-in-field	12
find-pcset-in-field	13
make-bounds-test.....	14
make-width-test	15
make-cardinality-test	16
make-spacing-test	17
make-voicing-test.....	18
make-p-vldg-test.....	19
make-pc-vldg-test	20
make-inclusion-test.....	21
and-tests.....	23
or-tests.....	24
filter-chordlist	25
sort-key_width	27
sort-key_incl	28
nesting-timing	29
incl-vec	30
sort-key_incl-vec-sum.....	31
incl-vec-angle.....	32
sort-key_incl-vec-angle.....	33
A. OMPF Annotated Bibliography.....	35

Chapter 1. Introduction

OMPitchField (OMPF for short) is a library for OpenMusic, designed to provide tools for composing with pitch fields. This Introduction gives an overview of the resources in OMPF and explains how to accomplish typical tasks with these tools. The accompanying OMPitchField Reference provides more detailed documentation of each function available in the library. Both documents assume the reader is familiar with a few simple terms and concepts from (American) musical set theory: pitch and pitch class (pc); pitch set and pcset; transposition and inversion of pcs and pitches; equivalence classes of pitch sets and pcsets based on transposition alone (Tn) or in combination with inversion (Tn/TnI); prime form of a Tn or Tn/TnI pcset class; and directed and undirected intervals in pitch and pc spaces. In its current form, OMPF deals exclusively with twelve-tone equal tempered pitch space and with the corresponding mod-12 pc space.

A pitch field is a construct that combines the properties and purposes of chord and scale: it is a collection of pitches assembled with regard for both its global harmonic qualities and its potential as a source of local harmonic and melodic detail. The global aspect of a pitch field, its identity as a chord, is made most explicit when all of its pitches are sounded together. But a field's characteristic harmonic sonority can also color other musical textures whose pitch content is limited to (or focused on) it, and different textures can be regarded as arpeggiating, animating, or sampling a pitch field in different ways. Once a pitch field's global identity has been sufficiently well established, a transition to a new field (possibly a transformation of the old) can create an area of contrast and help to articulate a composition's large-scale form.

As a scale, a pitch field imposes constraints on the pitch-space realizations of both contours and pcsets. On one hand, a field provides a space of generic scale-steps onto which contours can be mapped and in which they acquire particular pcset and pitch-set identities. And on the other hand, a pitch field limits the available pitch-space realizations of each pc and thus of each pcset. Or from another perspective, a pitch field limits the number and type of pcsets that can be expressed with a given registral disposition. Finally, whenever smaller local harmonies are drawn from a pitch field, the remaining field pitches provide material for melodic elaborations of these harmonies. (For more on the underlying theory of pitch fields, as well as a discussion relating this theory to the practice of various composers, see the author's essay "Pitch Fields: A Theory of Fixed-Pitch Formations," forthcoming in *Perspectives of New Music*.)

Most of the tools in OMPF are low-level or general enough to serve in a variety of applications. Thus while no attempt has been made to provide a complete range of "classic" pcset theory tools, composers and theorists may find OMPF helpful in accomplishing a variety of familiar but labor-intensive tasks. For instance, one of the patches in the Examples folder provided with OMPF illustrates how to construct and analyze all the 2-partitions of a pcset (see the patch "partitions").

At the same time, each function in OMPF plays a particular role in the process around which the library was mainly conceived. This process, which yields a pitch field and a compatible collection of smaller pitch sets (for both melodic and harmonic use), consists of four stages:

1. construct a pool of pcsets by filtering a large catalog according to special criteria
2. generate a pitch field from a cycle of directed pitch intervals
3. search for pitch-space realizations of each of pcset from the pool in the pitch field
4. filter the search results according to special criteria to produce a collection of pitch sets compatible with the field

Each OMPF function can be associated with one or more of these stages.

1.1. Stage 1. The pcset pool

The creation of a pcset pool in stage 1 need not involve OMPF resources - the pool can simply be a list of pcsets assembled by hand. But the technique of building and filtering a catalog is supported with special tools. The functions `list-t-primeforms` and `list-ti-primeforms` build lists containing the prime form of every T_n or T_n/T_{n1} pcset class for specified cardinalities, and the functions `make-t-setclass` and `make-ti-setclass`, which list all the members of a pcset class given any one member, can expand these prime-form lists into complete catalogs of pcsets (listing every member of every set class for specified cardinalities).

Various filters can be applied to either the prime-form list or the fully expanded pcset catalog. Filter the prime-form list when you can: it is small and filters quickly; also, the catalog will be smaller if it is built on a prime-form list that has already been filtered, and any additional filters applied to the catalog will run more quickly when the catalog is smaller. You may need to experiment with filter types and settings to produce the right size pool: you want it large enough for the searching and filtering in stages 3 and 4 to be productive, but small enough to keep the product focused.

The `filter-chordlist` function will filter a list of pcsets according to variable criteria, which should be expressed as a function that tests a chord and returns `t` or `nil`. Users can define their own test criteria using OpenMusic functions or subpatches in "lambda" mode. In addition, a number of predefined filter tests are provided: `make-cardinality-test`, `make-pc-vldg-test`, and `make-inclusion-test`. These tests can be combined logically with the `and-tests` and `or-tests` functions. With these resources you can, for example:

- pass all the pcset classes of cardinality 4 or greater that contain fewer than two members of the T_n pcset class (0 3 4) - use `and-tests` to combine one `make-cardinality-test` and one `make-inclusion-test`
- reject all the pcsets with more than two tones in common with the pcset (3 5 7 8 10), or less than two tones related by interval class 1 to the dyad (3 10) - use `or-tests` to combine two `make-pc-vldg-test` modules (this process is illustrated by the patch "or-tests" in the OMPF Examples folder)

You can also achieve the effect of filtering by sorting according to certain criteria and then selecting a region of the sort results. This is the preferred idiom when the criteria are not intuitively quantifiable. For instance, it is clear what you will get from a filter configured to pass sets with cardinalities in the range [3 , 5], but less clear what you will get from a filter configured to pass sets whose similarity to (0 2 3 5) falls in the range [0.785 , 1.047]. In place of this second (hypothetical) filter, use `sort-key_incl-vec-angle` with the OpenMusic `sort` function; then you can inspect the sort results and select the region you want using standard Common Lisp functions like `subseq`. The additional OMPF key functions `sort-key_incl` and `sort-key_incl-vec-sum` can also be used in this manner. (More information about the OMPF tools dealing with inclusion and similarity is collected at the end of this document.)

1.2. Stage 2. Cyclically generated pitch fields

In stage 2 of the process described above, we construct a pitch field. Any collection of pitches can serve as a field, but the OMPF function `make-pfield` provides access to the

special class of cyclically generated pitch fields. This is a large class, and an attractive one for reasons discussed at length in the essay "Pitch Fields" (cited above). Each field in this class unfolds (or is generated by) a cyclic interval pattern, which occurs repeatedly between adjacent pitches as we traverse the field from bottom to top. For example, the field (... -5 -4 0 1 5 6 10 11 15 ...) unfolds (or is generated by) the cycle (1 4). Assuming the cycle cannot be segmented exhaustively into copies of a smaller pattern (ruling out cycles like (1 3 1 3)), there will be M distinct transpositions of the field, where M is the sum of the intervals in the cycle. To invert a field, we simply reverse its generating cycle. When the interval sum M is not a factor of 12, the pitch-class content of the field will vary from one octave to the next, which is usually desirable. Cyclically generated pitch fields have a number of other nice properties, but the main reason they are built into the machinery of OMPF is their simplicity - users can experiment with the entire class simply by varying two parameters: the generating interval cycle and the transposition level.

1.3. Stage 3. Searching the field

The third of the four stages outlined above involves searching the pitch field for pitch-space realizations of each pcset from the pool. Before you launch a pcset search, it may be useful to examine the available pitch-space realizations of each individual pc. Connect the output of `make-pfield` to the second input (`field`) of `find-pc-in-field`, and connect the pc list (0 1 2 3 4 5 6 7 8 9 10 11) to the first input (`pc`) of the same function. You can evaluate this function directly and view its return value in the Listener window (as always), or connect its output to a chord-seq factory to see music notation. If you want notation, you will need to use the utility function `p->mc` to convert from OMPF's pitch space (middle-C = 0, semitone = 1 unit) to midicents (middle-C = 6000, semitone = 100 units). Each empty sublist in this result corresponds to a pc that is missing from the field. If on one hand there are many missing pcs, you may have unproductive pcset searches. Consider changing the pitch field's generating interval cycle or moving its truncation bounds further apart. If the interval cycle contains large values, consider replacing one or more of them with a group of smaller values having the same sum; then you will be adding new pitches to the field without removing any of the old ones. If on the other hand many of the sublists contain more than two or three values, then each pcset in the pool is likely to have a significant number of pitch-space realizations and searching for all of them will be slow.¹

1. A search in OMPF can be worse than slow: if the pool is large and the field contains multiple instances of most pcs, an "out of memory" error is likely to occur. If it does, simply choose "Abort" from the Lisp menu and try one or more of the following before repeating your search:

- If your pcset catalog is based on prime forms with more than one cardinality, try repeating the search for one cardinality at a time.
- Apply additional pcset filtering to the prime-form list or the pcset catalog to reduce the size of the search pool
- Select portions of the pool and process them one at a time
- Move the pitch field's truncation bounds closer together
- Adjust the field's generating cycle of intervals - for example, replace two adjacent values with their sum
- Delete individual pitches from the field by supplying a list of these pitches to the first input (pcset - a misnomer in this case) to `set-complement` and supplying the field to the optional second input (`space`). See the patch "pfield pruning" in the OMPF Examples folder.

The function `find-pcset-in-field` searches a pitch field and returns a list of all the instances it finds of a specified pcset X. This result will be all the possible pitch-space realizations X that

- are subsets of the pitch field, and
- exclude multiple instances of any pc;

the latter condition means that the resulting pitch sets will never contain octave duplications. In addition to searching for individual pcsets, you can search for a list of them at once with `find-pcset-in-field`, so searching for every item in a pcset pool is straightforward. Pools assembled with the tools described above will often be organized as nested lists - for example, a sublist for members of a particular ti-setclass, inside a sublist for ti-setclasses of a particular cardinality - in which case the search results will inherit the same structure, making it easy to trace portions back to particular pcsets or pcset families.

To view search results in music notation, you will have to take this nesting structure into account, because the pitch data supplied to a chord-seq factory must be structured in a particular way, as a list of flat lists of midicent values. The function `p->mc`, described above, handles the conversion to midicents. To eliminate excess levels of nesting, you can use the OM function `flat`, or a special OMPF function, `nesting->timing`, which behaves similarly but provides in addition a duration list that expresses the structure lost to flattening. For example, the pitch-set list

```
((((0 10) (10 12)) ((1 11) (11 13)))
 ((2 4 6) (4 6 14)) ((3 5 7) (5 7 15))))
```

can be flattened with `nesting->timing` to produce a list with the following elements and associated timings (assuming time-unit is set to 500 msec):

(0 10)	(10 12)	(1 11)	(11 13)	(2 4 6)	(4 6 14)	(3 5 7)	(5 7 15)
500	1000	500	1500	500	1000	500	2000

1.4. Stage 4. Processing the search results

The `filter-chordlist` function prescribed above for filtering lists of pcsets can also be used to filter lists of pitch sets. All of the filter tests used with pcset lists can be used with pitch-set lists too: `make-cardinality-test`, `make-pc-vldg-test`, and `make-inclusion-test`. In addition, there are several filter tests designed specifically for lists of pitch sets: `make-bounds-test`, `make-width-test`, `make-spacing-test`, `make-voicing-test`, and `make-p-vldg-test`. (See the OMPF Reference for details on each of these.) As explained above, filter tests can be combined with the `and-tests` and `or-tests` functions, and users can create additional tests using OpenMusic functions and subpatches in `?lambda?` mode.

You can also sort the search results. This is useful both to create the effect of filtering, via the `sort-and-select` idiom described above, and to organize search results for easy reference

during the course of composition. The following key functions are predefined in OMPF for use with the OpenMusic sort. function: `sort-key_width`, `sort-key_incl`, `sort-key_incl-vec-sum`, `sort-key_incl-vec-angle`. (Again, consult the OMPF Reference for more details.)

1.5. On inclusion and similarity

Several OMPF functions involve notions of inclusion and similarity. The basis for each of these is the inclusion vector, a broad generalization of Alan Forte's (1973) interval vector. The interval vector of a pcset X is a six-place vector in which the nth place (counting from 1) indicates the multiplicity of occurrence of interval class n in X. The inclusion vector of a pcset or pitch set Y is a similar construct: a vector of any length, in which each place counts the multiplicity of occurrence of members of a specified equivalence class in Y. An inclusion vector is built on a corresponding list of classrep expressions, each of which identifies an equivalence class of pcsets or pitch sets. The full classrep syntax - there are several possibilities - is specified in the OMPF Reference.

Here are some examples of classrep lists and corresponding inclusion vectors:

(1 2 3 4 5 6)	usual interval vector
((:ti 0 1 2) (:ti 0 1 3) (:ti 0 1 4) (:ti 0 1 5) (:ti 0 1 6) (:ti 0 2 4) (:ti 0 2 5) (:ti 0 2 6) (:ti 0 2 7) (:ti 0 3 6) (:ti 0 3 7) (:ti 0 4 8))	trichord vector (reports multiplicity of occurrence of each ti-setclass of cardinality
((:tp 0 10) (:tp 0 11) (:tp 0 13) (:tp 0 14))	reports multiplicity of occurrence of major and minor sevenths and ninths in a pitch set
(1 (:ti 0 1 2) (:ti 0 1 3 4) (:ti 0 1 4 5) (:ti 0 1 5 6) (:ti 0 1 6 7))	hybrid vector (these can be informative in a particular compositional context; see the "dissonance" patch in the OMPF Examples folder)

In the second of these examples, the classrep list is a complete listing of Tn/Tnl prime forms of cardinality 3. The function `list-ti-primeforms` can construct such a list - complete with the `:ti` tags - automatically.

Several OMPF functions involve inclusion vectors. The function `incl-vec` computes the inclusion vector itself. The key function `sort-key_incl-vec-sum` allows a list of chords to be sorted based on a weighted sum of their inclusion-vector entries; a procedure based on this tool, for sorting chords roughly in order of increasing dissonance, is illustrated in the OMPF Examples folder ("dissonance" patch). Other OMPF functions, namely `make-inclusion-test` and `sort-key_incl`, take a single classrep expression and compute the multiplicity of occurrence of members of the relevant equivalence class as a basis for filtering or sorting.

Finally, OMPF incorporates the idea of "interval angle" proposed by Damon Scott & Eric J. Isaacson as a measure of pcset similarity.² The interval angle from one pcset to another is calculated by situating their interval vectors at a common origin in six-dimensional space

2. Scott & Isaacson (1998) "The Interval Angle: A Similarity Measure for Pitch-Class Sets," *Perspectives of New Music* 36: 107-142.

and computing the angle (a value between 0 and $\pi/2$ radians) from one vector to the other. The smaller this angle, the more "intervallically similar" the pcsets can be said to be. In OMPF, the "interval angle" is generalized to involve n-dimensional computations with any conceivable type of inclusion vector. See the patch "angle" in the OMPF Examples folder for examples of its use. (If you intend to compute interval angles using nonstandard inclusion vectors, note the caveats stated in the entry for `incl-vec-angle` in the OMPF Reference.) In addition to computing interval angles, you can sort a list of chords based on inclusion-vector angle measures between each chord and a fixed reference chord with the function `sort-key_incl-vec-angle`.

1.6. Where to go from here

If the underlying set-theory principles are familiar and you find this introductory presentation sufficiently clear, you should be equipped to use OMPF tools freely, consulting the OMPF Reference as needed. Otherwise, study the patches in the OMPF Examples folder, especially the one called "4 stages" which implements the main process for which this library was originally conceived. In this patch, experiment with changing parameters in each stage:

- filter test types and settings used in the pcset pool
- pcset cardinalities included in the pool
- pitch field generator and origin
- test types and settings for filtering the search results

In the OMPF Examples folder you will also find the following patches (listed with the main OMPF functions they illustrate):

angle: uses functions `list-ti-primeforms` `incl-vec-angle`

dissonance: uses functions `list-ti-primeforms` `sort-key_incl-vec-sum` `find-pcset-in-field` `nesting->timing`

or-tests: uses functions `make-ti-setclass` `nesting->timing` `make-pc-vldg-test` `or-tests` `filter-chordlist`

pfield pruning: uses functions `set-complement` `make-pfield`

partitions: uses functions `list-ti-primeforms` `make-ti-setclass` `find-pcset-in-field` `set-complement` `t-primeform` `nesting->timing`

I. OMPitchField Reference

list-ti-primeforms

Name

list-ti-primeforms — print formatted output

Inputs

<i>card</i>	cardinality of desired primeforms	primeforms integer in [2 , 10] (or a list of such numbers, to produce results for more than one cardinality)
<i>tag</i>	option to insert tag :ti at head of each primeform	menu selection: nil or :ti

Output

list of all ti-primeforms of specified cardinality	list (or list of lists) of pcsets (each pcset a list of mod12 integers, with optional tag :ti at head)
--	--

Description

A `ti-setclass` is a family of pcsets related to one another by transposition and/or inversion, and its `ti-primeform` is a member of the family, designated to represent the entire family.

This function lists the designated representative member of every `ti-setclass` with the specified cardinality *card*. This input parameter, normally an integer, can also be a list, to produce results for more than one cardinality. The "tag" option facilitates the construction of the *classreps* parameter required as an input to several functions in OMPitchField (see details in the description of `make-inclusion-test`).

list-t-primeforms

Name

list-t-primeforms — print formatted output

Inputs

<i>card</i>	cardinality of desired primeforms	integer in [2 , 10] (or a list of such numbers, to produce results for more than one cardinality)
<i>>tag</i>	option to insert tag :ti at head of each primeform	menu selection: nil or :t

Output

list of all t-primeforms of specified cardinality	list (or list of lists) of pcsets (each pcset a list of mod12 integers, with optional tag :t at head)
---	---

Description

A `t-setclass` is a family of pcsets related to one another by transposition, and its t-primeform is a member of the family, designated to represent the entire family.

This function lists the designated representative member of every t-setclass with the specified cardinality *card*. This input parameter, normally an integer, can also be a list, to produced results for more than one cardinality. The "tag" option facilitates the construction of the *classreps* parameter required as an input to several functions in this library (see function `make-inclusion-test`).

t-primeform

Name

t-primeform — outputs pcset close to zero

Inputs

<i>pcset</i>	any member of the t-setclass whose primeform is sought (or a list of these)	list (or list of lists) of mod12 integers
--------------	---	---

Output

primeform of the t-setclass to which <i>pcset</i> belongs (or a list with the primeform of each input <i>pcset</i>)	list (or list of lists) of mod12 integers
--	---

Description

A t-setclass is a family of pcsets related to one another by transposition, and its t-primeform is a member of the family, designated to represent the entire family. The t-primeform algorithm selects the *pcset* whose elements are maximally close to zero in a particular sense.

ti-primeform

Name

ti-primeform — outputs pcsets close to zero

Inputs

<i>pcset</i>	any member of the ti-setclass whose primeform is sought (or a list of these)	list (or list of lists) of mod12 integers
--------------	--	---

Output

primeform of the ti-setclass to which <i>pcset</i> belongs (or a list with the primeform of each input <i>pcset</i>)	list (or list of lists) of mod12 integers
---	---

Description

A ti-setclass is a family of *pcset* related to one another by transposition, and its *ti-primeform* is a member of the family, designated to represent the entire family. The ti-primeform algorithm selects the *pcset* whose elements are maximally close to zero in a particular sense. This algorithm is equivalent to one introduced by John Rahn (1980) and adopted by Joseph Straus and Robert Morris among others; Allen Forte's original prime form algorithm (1973) produces different results in a small number of cases. (Essentially, the Rahn prime form minimizes the number of large pc integers, while the Forte prime form minimizes the largest one and then maximizes the number of small ones.)

make-t-setclass

Name

make-t-setclass — lists t-setclass members

Inputs

<i>pcset</i>	any member of the desired t-setclass (or list containing one member from each desired t-setclass)	list (or list of lists) of mod12 integers (or a list of such lists)
--------------	---	---

Output

family of pcsets containing <i>pcset</i> and all of its transpositions (multiple families, when multiple <i>pcsets</i> are input)	list (or list of lists) of mod12 integers(or list of lists of lists of mod-12 integers)
---	---

Description

Given any member of a t-setclass, lists every member of that t-setclass.

make-ti-setclass

Name

`make-ti-setclass` — lists ti-setclass members

Inputs

<i>pcset</i>	any member of the desired ti-setclass (or list containing one member from each desired ti-setclass)	list (or list of lists) of mod12 integers (or a list of such lists)
--------------	---	---

Output

family of pcsets containing <i>pcset</i> its inversion (at index 0), and all of their transpositions (multiple families, when multiple pcsets are input)	list of lists of mod-12 integers (or list of lists of lists of mod-12 integers)
--	---

Description

Given any member of a ti-setclass, lists every member of that `ti-setclass`.

set-complement

Name

set-complement — outputs pcset complement

Inputs

<i>pcset</i>	<i>pcset</i> whose complement is sought (or list of <i>pcsets</i>)	list (or list of lists) of mod12 integers (or a list of such lists)
--------------	---	---

Optional Input

<i>space</i>	pitch class space in which complementation is performed (by default, the full chromatic aggregate)	list of mod12 integers
--------------	--	------------------------

Output

the complement of <i>pcset</i> in space (i.e. the set of all elements in space that are not members of <i>pcset</i>)	list of mod-12 integers (or list of such lists)
---	---

Description

Computes the complement of the input *pcset* (*s*) with respect to the full chromatic aggregate or to another space when specified.

make-pfield

Name

make-pfield — generates a pitch field

Inputs

<i>gen</i>	cyclic generator (cycle of intervals from which pfield is generated)	list of integers
<i>origin</i>	pitch in <i>pfield</i> coinciding with start of the cyclic generator <i>gen</i>	integer (or a list of such lists)
<i>lo</i>	pitch below which <i>pfield</i> is truncated	integer
<i>hi</i>	pitch above which <i>pfield</i> is truncated	integer

Output

<i>pfield</i>	space of pitches that unfolds the cyclic generator <i>gen</i>	list of pitches
---------------	---	-----------------

Description

A pitch field is a space of pitches that repeatedly unfolds (or is generated by) a cyclic interval pattern. For instance, the field

(... -4 0 1 5 6 10 11 15 ...)

unfolds the cycle

(1 4)

Assuming the cycle cannot be partitioned exhaustively into copies of a smaller pattern (ruling out cycles like

(4 1 4 1))

, there will be M distinct transpositions of the field, where M is the sum of the generating intervals. Pitch fields, which extend infinitely low and high in theory, retain most of their interesting properties when truncated for use in composition. For more on the underlying theory, see the author's article "Pitch Fields: A Study of Fixed-Pitch Formations" (to appear in *Perspectives of New Music*).

This function generates a pitch field pfield with generator *gen*, transposed so that a cycle of *gen* begins at *origin*, and truncated between *lo* and *hi*.

p->mc

Name

p->mc — converts p to mc

Inputs

p	a pitch-space value or list of them	integer or list of integers
-----	-------------------------------------	-----------------------------

Output

mc	a midicents value or list of them	integer or list of integers
------	-----------------------------------	-----------------------------

Description

Converts from pitch-space values to midicents values

- pitch space: middle-C = 0, semitone = 1
- midicents: middle-C = 6000, semitone = 100

mc->p

Name

mc->p — converts mc to p

Inputs

<i>mc</i>	a midicents value or list of them	integer or list of integers
-----------	-----------------------------------	-----------------------------

Output

<i>p</i>	a pitch-space value or list of them	integer or list of integers
----------	-------------------------------------	-----------------------------

Description

Converts from midicents values to pitch-space values

- pitch space: middle-C = 0, semitone = 1
- midicents: middle-C = 6000, semitone = 100

find-pc-in-field

Name

find-pc-in-field — pitch field search

Inputs

<i>pc</i>	a pitch class or list of them	mod12 integer or list of them
<i>field</i>	a pitch field (any collection of pitches)	list of integers

Output

list of all the pitches in <i>field</i> that are congruent mod12 to <i>pc</i> (or list containing one such list for each input pitch class)	list of mod12 integers (or list of lists of mod12 integers)
---	---

Description

Searches a pitch field (which can be a cyclically generated space of the type produced by `make-pfield`, or any other collection of pitches) and returns all the instances it finds of the pitch class *pc* (or of each pitch class, if *pc* is a list of them).

find-pcset-in-field

Name

find-pcset-in-field — pitch field search

Inputs

<i>pcset</i>	a pitch class set or list of them	them list of mod12 integers, or list of lists of mod12 integers
<i>field</i>	a pitch field (any collection of pitches)	list of integers

Output

list of all the pitch sets in <i>field</i> that are pitch-space realizations of <i>pcset</i> (or list containing one such list for each input pitch class set)	list of lists of mod12 integers (or list of lists of lists of mod12 integers)
--	---

Description

Searches a pitch field (which can be a cyclically generated space of the type produced by `make-pfield`, or any other collection of pitches) and returns all the instances it finds of the pitch class *pcset* (or of each pitch class, if *pcset* is a list of them).

make-bounds-test

Name

make-bounds-test — test chord interval

Inputs

<i>lo</i>	lower bound of pitch interval	integer
<i>hi</i>	upper bound of pitch interval	integer

Output

test that will return t or nil	compiled lexical closure
--------------------------------	--------------------------

Description

Returns a predicate, intended for use with `filter-chordlist` (see the description of this function), to test if a chord fits entirely within the closed interval [*lo* , *hi*].

Users do not need to set this function to "lambda" mode using **OpenMusic**'s button feature; it operates in this mode by default.

make-width-test

Name

make-width-test — test chord registral span

Inputs

<i>width</i>	size of pitch interval	integer
--------------	------------------------	---------

Output

test that will return t or nil	compiled lexical closure
--------------------------------	--------------------------

Description

Returns a predicate, intended for use with `filter-chordlist` (see the description of this function), to test if the registral span of a chord (the interval from its lowest to highest note) is less than or equal to *width*.

Users do not need to set this function to "lambda" mode using **OpenMusic**'s button feature; it operates in this mode by default.

make-cardinality-test

Name

make-width-test — test chord notes

Inputs

<i>lo</i>	lower bound on cardinality	integer
<i>hi</i>	upper bound on cardinality	integer

Output

test that will return t or nil	compiled lexical closure
--------------------------------	--------------------------

Description

Returns a predicate, intended for use with `filter-chordlist` (see the description of this function), to test if the number of notes in a chord is at least *lo* and at most *hi*.

Users do not need to set this function to "lambda" mode using **OpenMusic**'s button feature; it operates in this mode by default.

make-spacing-test

Name

make-spacing-test — test note distances in chord

Inputs

<i>spacing-pairs</i>	list of specifications for chord spacing	spacing list of pairs; each pair is a list of two integers, the second no smaller than the first
----------------------	--	--

Output

test that will return t or nil	compiled lexical closure
--------------------------------	--------------------------

Description

Returns a predicate, intended for use with `filter-chordlist` (see the description of this function), to test if the distances between consecutive notes of a chord, traversed from bottom to top, are in the intervals specified by corresponding elements of `spacing-pairs`. Each element of this list is a pair (*lo hi*) specifying the bounds of a closed interval. If there is a surplus of `spacing-pairs`, ignores the extras; if there is a shortage, re-uses the last.

Users do not need to set this function to "lambda" mode using **OpenMusic**'s button feature; it operates in this mode by default.

make-voicing-test

Name

make-voicing-test — tests on specific interval classes

Inputs

<i>vspec-pairs</i>	list of specifications for chord voicing	list of pairs; each pair is a list of the form (<i>ints</i> <i>lim</i>); each <i>ints</i> is a list of integers; each <i>lim</i> is a number between -1 and 1
--------------------	--	---

Output

test that will return t or nil	compiled lexical closure
--------------------------------	--------------------------

Description

Returns a predicate, intended for use with `filter-chordlist` (see the description of this function), to test if specific interval classes (undirected *pc* intervals) in a chord are voiced according to the criteria specified in *vspec-pairs*, which is a list of pairs (*ints* *lims*). Each *ints* is a list of undirected pitch intervals drawn from a single interval class -- e.g. (11 13) --, and the absolute value of the corresponding *lim*, a positive (negative) number between 0 and 1 (-1), determines a lower (upper) limit for the ratio *J*:*K*, where *K* is the multiplicity of occurrence of the interval class represented in *ints*, and *J* is the combined multiplicity of occurrence of the undirected pitch intervals listed in *ints*.

Examples: with *vspec-pairs* = (((10 22) 3/4)), returns a test to see if at least 3/4 of the instances of interval class 2 in a chord are voiced as pitch interval 10 or 22. With *vspec-pairs* = (((1) 1/6) ((1) -1/2)), returns a test to see if at least 1/6, but at most 1/2, of the instances of interval class 1 in a chord are voiced as undirected pitch interval 1.

Users do not need to set this function to "lambda" mode using **OpenMusic**'s button feature; it operates in this mode by default.

make-p-vldg-test

Name

make-p-vldg-test — tests total voiceleading

Inputs

<i>from-chord</i>	chord from which <i>voice-leading</i> intervals depart	list of integers
<i>vlspec-triples</i>	list of specifications for a chord in terms of its <i>voice-leading</i> from from-chord	list of triples; each triple is a list of the form (<i>ints lo hi</i>); each <i>ints</i> is a list containing one or more integers; <i>lo</i> is an integer, and <i>hi</i> is an integer no smaller than <i>lo</i>

Output

test that will return t or nil	compiled lexical closure
--------------------------------	--------------------------

Description

Returns a predicate, intended for use with `filter-chordlist` (see the description of this function), to test a chord in terms of the total voiceleading to it from *from-chord*. The total voiceleading $VL(X,Y)$ is the multiset of all directed intervals from elements of the proper set X to elements of the proper set Y . The predicate described here tests a chord C by checking the multiplicity of specific intervals in

$V = VL(\text{from-chord}, C)$ against the specifications in *vlspec-triples*.

The *vlspec-triples* parameter is a list of triples (*ints lo hi*). Each *ints* is a list containing one or more directed pitch intervals, and the corresponding *lo* and *hi* specify lower and upper limits on the combined multiplicity of occurrence in V of the intervals from *ints*.

Users do not need to set this function to "lambda" mode using **OpenMusic's** button feature; it operates in this mode by default.

make-pc-vldg-test

Name

make-pc-vldg-test — tests total voiceleading

Inputs

<i>from-chord</i>	chord (conceived as either pcset or pitch set) from which voice-leading intervals depart	list of integers (mod12 or unrestricted)
<i>vlspec-triples</i>	list of specifications for a chord in terms of its <i>voice-leading</i> from <i>from-chord</i>	list of triples; each triple is a list of the form (<i>pci lo hi</i>); each <i>pci</i> is an integer in [-6 , 11]; <i>lo</i> is an integer, and <i>hi</i> is an integer no smaller than <i>lo</i>

Output

test that will return t or nil	compiled lexical closure
--------------------------------	--------------------------

Description

Operates exactly like make-p-vldg-test, but

- *from-chord* may be a pcset or a pitch set
- the *vlspec-triples* have the form (*pci lo hi*), where *pci* is a directed or undirected (possibly zero) pitch class interval
- the *voice-leading* $V = VL(\text{from-chord}, C)$ is reckoned in terms of directed or undirected pitch class intervals

Each *pci* value x is interpreted as follows: if x is nonpositive, then its absolute value is understood to be an undirected pitch class interval (i.e. an interval class) in $[0, 6]$; if x is nonnegative, then it is understood to be a directed pitch class interval in $[0, 11]$.

make-inclusion-test

Name

make-inclusion-test — returns a predicate

Inputs

<i>incspec-triples</i>	list of specifications for a chord in terms of inclusion relations	list of triples; each triple is a list of the form (<i>classrep lo hi</i>); <i>lo</i> and <i>hi</i> are integers; and each <i>classrep</i> has one of the structures described in the following chart:
------------------------	--	--

classrep parameter

interval class, possibly 0	integer in [0 , 6]
t-setclass (family of pcsets)	list with head :t and tail of mod12 integers
ti-setclass (family of pcsets)	list with head :ti and tail of mod12 integers
t-class of pitch sets	list with head :tp and tail of mod12 integers
ti-class of pitch sets	list with head :tip and tail of mod12 integers

Output

test that will return t or nil	compiled lexical closure
--------------------------------	--------------------------

Description

Returns a predicate, intended for use with `filter-chordlist` (see the description of this function), to test a chord in terms of the subsets it includes (and/or the supersets that include it). The *incspec-triples* specify classes built on transposition alone or with inversion, of pcsets or pitch sets; they also specify limits on the number of instances of inclusion that occur between each of these classes and the chord *C* to be tested.

If *C* is larger than a particular classrep *CR*, then the predicate checks if the multiplicity of occurrence in *C* of members of the class defined by *CR* is within [*lo* , *hi*]. If *C* is smaller than *CR*, then the predicate checks if the number of members of the class defined by *CR* that include *C* is within [*lo* , *hi*].

Users do not need to set this function to "lambda" mode using **OpenMusic**'s button feature; it operates in this mode by default.

and-tests

Name

and-tests — returns a test

Inputs

Inputs vary. Any number of test items:

<i>test</i>	predicate function	closure
-------------	--------------------	---------

Output

test that will return t or nil	compiled lexical closure
--------------------------------	--------------------------

Description

Takes any number of predicate functions (each returning t or nil) and returns a test that, for a certain input, will return t if all of the predicates return t for the same input, or nil if any of the predicates return nil.

Each predicate function test may be an **OpenMusic** function or patch in "lambda" mode, or one of the **OMPitchField** test functions like make-bounds-test.

or-tests

Name

or-tests — returns a test

Inputs

Inputs vary. Any number of test items:

<i>test</i>	predicate function	closure
-------------	--------------------	---------

Output

test that will return t or nil	compiled lexical closure
--------------------------------	--------------------------

Description

Takes any number of predicate functions (each returning t or nil) and returns a test that, for a certain input, will return t if one or more of the predicates return t for the same input, or nil if all of the predicates return nil.

Each predicate function test may be an **OpenMusic** function or patch in "lambda" mode, or one of the **OMPitchField** test functions like `make-bounds-test`.

filter-chordlist

Name

filter-chordlist — filters a list of chords

Inputs

<i>test</i>	predicate function that tests a chord (list of integers) and returns t or nil	function
<i>chordlist</i>	list (possibly nested) of chords	list (possibly nested) of lists of integers
<i>reject</i>	expression representing an equivalence class of pcs or pitches	<i>menu selection: pass</i> or <i>reject</i>

Output

filtered-chordlist	list (possibly nested) of chords	list (possibly nested) of lists of integers
--------------------	----------------------------------	---

Description

The *test* parameter should be a predicate function returning t or nil when applied to each chord in the (nested) list *chordlist*. The return value *filtered-chordlist* is a list like *chordlist* but with certain chords removed. If the third input is set to *pass*, then chords for which *test* returns t will be passed through to *filtered-chordlist* and the others removed. If the third input is set to *reject*, then chords for which *test* returns t will be removed and the others passed through to *filtered-chordlist*.

Note: this function performs as intended only when the input *chordlist* is a list with a particular structure S, which if not empty can contain chords (nonempty integer lists) or lists with structure S, but not a mix of the two.

Examples (each item chord-n is a nonempty list of integers):

- good: ((chord-1 chord-2) (chord-3) () (chord-4 chord-5))
- good: (chord-1 chord-2 chord-3)
- good: ((chord-1 chord-2) (() (chord-3 chord-4) (chord-5)))
- bad: (chord-1 chord-2 nil chord-3)

list contains a mix of chords and S structures

- `bad: ((chord-1 (chord-2 chord-3)) (chord-4 chord-5))`
first sublist contains a mix of chords and S structures

For other filtering tasks, consider the **Common Lisp** functions `remove-if` and `remove-if-not`, and **OpenMusic** functions like `filter-list`.

sort-key_width

Name

sort-key_width — returns a function

Output

test that will return t or nil	compiled lexical closure
--------------------------------	--------------------------

Description

Returns a function that assigns a number to a chord according to the chord's width. Intended for use as a key function with the **OpenMusic** `sort.` function, in order to sort a list of chords in order of increasing or decreasing registral span.

Users do not need to set this function to "lambda" mode using **OpenMusic**'s button feature; it operates in this mode by default.

sort-key_incl

Name

sort-key_incl — returns a function

Inputs

<i>classrep</i>	expression representing an equivalence class of pcs or pitches	<i>for details, see function make-inclusion-test</i>
-----------------	--	--

Output

test that will return t or nil	compiled lexical closure
--------------------------------	--------------------------

Description

Returns a function that assigns a chord the number of members of a specified equivalence class (particular interval class, t- or ti-setclass of pcs, or t- or ti-setclass of pitches, as specified by classrep) that include or are included in the chord. Intended for use as a key function with the **OpenMusic** sort. function, in order to sort a list of chords in order of increasing or decreasing equivalence-class content.

Users do not need to set this function to "lambda" mode using **OpenMusic**'s button feature; it operates in this mode by default.

nesting-timing

Name

nesting-timing — flattens a nested list

Inputs

<i>lst</i>	a list to be flattened	nested list
<i>level</i>	the number of nested levels to flatten (or the number to remain after flattening)	positive integer (or nonpositive integer with special interpretation)
<i>time-unit</i>	default interval (in msec) between onset times in output <i>timing-list</i>	positive integer

Output

<i>flattened-list</i>	<i>lst</i> with some or all of its nesting structure removed by flattening	list
<i>timing-list</i>	list of onset times separated by <i>time-unit</i> , or by a multiple of this amount when the separation corresponds to a boundary between sublists of <i>lst</i> that have been joined by flattening	list

Description

The intended use of this function is to flatten a nested list of pitch data into a form acceptable to a chord-seq factory, and to recover the structure lost to flattening in the form of timing data acceptable to the factory. In patches, the usual wiring will be: *flattened-list* to chord-seq input 2, *timing-list* to chord-seq input 3.

If *levels* is a positive number *l*, then *l* levels of nesting will be removed in *flattened-list*. If *levels* is a nonpositive number *m*, then all but *|m|* levels of nesting will be removed in *flattened-list*; thus setting *levels* to zero flattens *lst* completely.

incl-vec

Name

`incl-vec` — specifies any idiosyncratic mix of equivalence classes

Inputs

<i>chord</i>	<i>chord</i> whose inclusion vector is sought (or a list of <i>chords</i>)	list of integers (or list of lists of integers)
<i>classreps</i>	list of expressions representing equivalence classes of pcs or pitches	<i>for details, see function</i> <code>make-inclusion-test</code>

Output

inclusion-vector reporting selected inclusion relations of chord	list of nonnegative integers
--	------------------------------

Description

The inclusion vector is a broad generalization of the familiar interval vector from which it derives its name.¹ For a given list of equivalence classes (particular interval classes, *t*- or *ti*-setclasses of pcs, and/or *t*- or *ti*-setclasses of pitches, as specified in *classreps*), the inclusion vector of *chord* is a corresponding list indicating how many members of each equivalence class contain or are contained by *chord*.

If *classreps* is a list representing all of the (nonzero) interval classes -- namely (1 2 3 4 5 6) -- then the inclusion vector is the interval vector whose uses have been discussed at length in the music-theory literature.

If *classreps* is an ordered list representing all of the *t*- or *ti*-primeforms of a particular cardinality -- for example, a list beginning ((:ti 0 1 2) (:ti 0 1 3) ...) for cardinality-3 *ti*-primeforms -- then the inclusion vector provides compact information about a chord's embedded subsets (or embedding supersets). A *classreps* list suitable for this purpose can be constructed automatically by the functions `list-t-primeforms` and `list-ti-primeforms` (with the "tag" option).

More generally, *classreps* may specify any idiosyncratic mix of equivalence classes that interests the user.

1. See Forte, *The Structure of Atonal Music* (1973), on interval vectors. Lewin's M-class vector (*Generalized Musical Intervals and Transformations*, 1987) is similar to (but more focused than) our inclusion-vector generalization. Despite the name, inclusion vectors are implemented in **OMPitchField** as lists, not vectors.

sort-key_incl-vec-sum

Name

sort-key_incl-vec-sum — returns a function

Inputs

<i>classreps</i>	list of expressions representing equivalence classes of pcs or pitches	<i>for details, see function make-inclusion-test</i>
<i>weightlist</i>	weighting applied to inclusion vector	list of numbers, one for each <i>classreps</i> item

Output

test that will return a number	compiled lexical closure
--------------------------------	--------------------------

Description

Returns a function that assigns a number to a chord according to a weighted sum of the positions in a chord's inclusion vector calculated for the equivalence-classes represented in *classreps*. The function returned by sort-key_incl-vec-sum is intended for use as a key function with the **OpenMusic** sort. function. For details on how inclusion vectors are computed, see function incl-vec. For details on the construction of a *classreps* list, see function make-inclusion-test.

In one straightforward application, *classreps* is the list (1 2 3 4 5 6), so the inclusion vector is the familiar interval vector. With weightings that reflect the potential dissonance of each interval class, this application allows a list of chords to be sorted roughly in order of increasing or decreasing dissonance.

incl-vec-angle

Name

`incl-vec-angle` — calculates inclusion vectors

Inputs

<i>chord1</i>	a chord	list of integers
<i>chord2</i>	a chord	list of integers
<i>classreps</i>	list of expressions representing equivalence classes of pcs or pitches	<i>for details, see function make-inclusion-test</i>

Output

the geometric angle (in radians) from the inclusion vector of <i>chord1</i> to that of <i>chord2</i>	real number in $[0, \pi/2]$
---	-----------------------------

Description

This function calculates inclusion vectors for *chord1* and *chord2* in terms of the *n* equivalence classes identified in *classreps*. It then situates these vectors at a common origin in *n*-dimensional space and computes the angle from the *chord1* vector to the *chord2* vector. For details on how inclusion vectors are computed, see function `incl-vec`. For details on the construction of a *classreps* list, see function `make-inclusion-test`.

When *classreps* is (1 2 3 4 5 6), the inclusion vectors are interval vectors, and the return value is the interval angle proposed by Scott & Isaacson (1998) as a measure of pcset similarity (smaller angles indicating greater similarity).

If *chord1* and/or *chord2* contains (or is contained by) zero members of all of the equivalence classes specified in *classreps*, then it will have a zero-magnitude vector -- a string of all zeros (0 0 ...) -- and a true angle measure is not defined. To preserve the utility of this function as a generalized similarity measure, the following solution is adopted in the zero-magnitude case. If both vectors have zero magnitude, the angle reported is zero (for maximum similarity); if one vector has zero magnitude and the other does not, the angle reported is $\pi/2$ (maximum dissimilarity). To avoid zero-magnitude vectors and ensure a result based on true angle measures, include in *classreps* the interval class 0, or all of the interval classes (1 2 3 4 5 6), or one of the complete primeform lists produced by the functions `list-t-primeforms` or `list-ti-primeforms` (with the "tag" option).

sort-key_incl-vec-angle

Name

sort-key_incl-vec-angle — returns a function

Inputs

<i>classreps</i>	list of expressions representing equivalence classes of pcs or pitches	<i>for details, see function make-inclusion-test</i>
<i>ref-chord</i>	chord whose inclusion-vector provides a reference from which angles are measured	list of integers

Output

test that will return a number	compiled lexical closure
--------------------------------	--------------------------

Description

Returns a function that assigns a number to a chord *c* based on the angle measured from the inclusion vector of *ref-chord* to the inclusion vector of *c*. The function returned by `sort-key_incl-vec-angle` is intended for use as a key function with the **OpenMusic** `sort.` function. For details on how inclusion vectors are computed, see function `incl-vec`. For details on the construction of a *classreps* list, see function `make-inclusion-test`. And for details on the calculation of inclusion-vector angles, see function `incl-vec-angle`.

Because the angle measure between the inclusion vectors of two chords is often plausibly interpreted as a measure of how similar the chords are (with smaller angles indicating greater similarity), this function allows a list of chords to be sorted in order of increasing or decreasing similarity to *ref-chord*.

Appendix A. OMPF Annotated Bibliography

For many concepts of musical set theory - as it is practiced in the United States, at least - the standard source remains

Forte, Allen. 1973. *The Structure of Atonal Music*. Yale University Press.

The theory of *set complexes* set forth in Forte's book is not explicitly implemented in **OMPF**, although the tools provided make this a fairly straightforward task. See the "Forte" folder, inside the OMPF Examples folder, for an implementation of Forte's later theory of pcset genera, which are closely related to set complexes. This is a relatively specialized topic, not of interest to every reader.

Forte, Allen. 1988. *Pitch-Class Set Genera and the Origin of the Modern Harmonic Species*. *Journal of Music Theory* 32.2: 187-270.

Other good introductory presentations of musical set theory include:

- Rahn, John. 1980. *Basic Atonal Theory*. Longman.
- Straus, Joseph. 1990. *Introduction to Post-Tonal Theory*. Prentice Hall.
- Morris, Robert. 1991. *Class Notes for Atonal Theory*. Frog Peak Music.
- -----, 2001. *Class Notes for Advanced Atonal Theory*. (2 vols.) Frog Peak Music.

Perhaps most relevant of all, for its explicitly compositional orientation and its proximity to the language and concepts in OMPF, is

- Morris, Robert. 1987. *Composition with Pitch Classes: A Theory of Compositional Design*. Yale University Press.

Other important treatments of musical set theory include, in increasing order of distance from the conceptual world of OMPF:

- Lewin, David. 1987. *Generalized Musical Intervals and Transformations*. Yale University Press.
- Mazzola, Guerino. 1990. *Geometrie der Töne: Elemente der Mathematischen Musiktheorie*. Birkhäuser.
- -----, Forthcoming (2002). *The Topos of Music*. Birkhäuser.
- Vieru, Anatol. 1993. *The Book of Modes*. Revised edition in English. Editura Muzicala. (Originally published in Romanian, 1980.)
- Schat, Peter. 1993. *The Tone Clock*. (Contemporary Music Studies, vol. 7.) Harwood Academic.

OMPF's inclusion-vector angle feature generalizes an idea in

Scott, Damon and Isaacson, Eric J. *The Interval Angle: A Similarity Measure for Pitch-Class Sets*. *Perspectives of New Music* 36: pp.107-142.

On pitch fields more specifically, see

Nauert, Paul. Forthcoming (2002). *Field Notes: A Theory of Fixed-Pitch Formations*. *Perspectives of New Music*.

The use of pitch-field techniques by specific composers is documented in a number of sources, including:

Elliott Carter

Link, John F. 1994. *The Composition of Elliott Carter's Night Fantasies*. Sonus 14.2: pp.67-89.

Mead, Andrew. 1995. *Twelve-Tone Composition and the Music of Elliott Carter*. In Elizabeth West Marvin and Richard Hermann, *Concert Music, Rock, and Jazz since 1945: Essays and Analytic Studies*. University of Rochester Press.

Schiff, David. 1998. *The Music of Elliott Carter*. 2nd ed. Cornell University Press. (1st ed. Da Capo Press, 1983.)

a number of additional Carter studies are cited in the items listed above

Witold Lutoslawski

Rae, Charles Bodman. 1994. *The Music of Lutoslawski*. Faber and Faber.

Stucky, Stephen. 1981. *Lutoslawski and His Music*. Cambridge University Press.

Mel Powell

Brandt, Anthony Kroyt. 1988. *The Absence of Nouns: Mel Powell's Harmonic Language*. *Aperiodical* 2.1: pp.42-44.

Melbinger, Timothy. 1999. *An Analysis of Mel Powell's Piano Trio '94*. Brandeis University dissertation.

Finally, **OpenMusic** users will want to explore the **Zn** classes and functions (in the OM kernel), which implement certain aspects of pcset theory with greater rigor and abstraction than are found in **OMPF**; there is a reasonable degree of compatibility between **Zn** and **OMPF** at present, and more is planned.

Andreatta, Moreno; Agon, Carlos; Chemillier, Marc. 1999. *OpenMusic et le problème de la construction de canons musicaux rythmiques*. Actes des 6èmes Journées d'Informatique Musicale, CNET-CEMAMu, pp.179-185. (Available online at <http://www.ircam.fr/equipes/repmus/RMPapers/>)